Лабораторная работа 6 "Работа с API браузера и асинхронными операциями"

Лабораторная работа №6. Работа с API браузера и асинхронными операциями

1. ЦЕЛЬ РАБОТЫ

Освоение принципов асинхронного программирования в JavaScript, работы с Fetch API, форматами JSON и клиентским хранением данных. Формирование навыков взаимодействия с внешними API и организации локального хранения состояния приложения на примере интеграции стороннего API, тематически связанного с вашим будущим курсовым проектом (2 часть курса).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы JavaScript (переменные, функции, объекты, массивы).
- Работу с DOM и обработку событий (из лабораторной работы №5).
- Понятие асинхронности и цикл событий в JavaScript.
- Базовые принципы HTTP-запросов.
- Структуру данных JSON (формат данных, которые возвращают большинство API).
- Основы безопасности веб-приложений (хранение API-ключей, CORS, HTTPS).

---

3. ЗАДАЧИ РАБОТЫ

- Освоить работу с `Promise`, `async/await` и Fetch API.
- Выполнять HTTP-запросы (GET, POST) к внешнему API, соответствующему тематике курсового проекта.
- Реализовать обработку данных в формате JSON.
- Организовать клиентское хранение данных с использованием `LocalStorage` (кэширование, избранное, история).
- Реализовать обработку ошибок при асинхронных операциях (сеть, неверный ответ, отсутствие данных).
- Интегрировать полученные данные в существующий интерфейс (созданный в предыдущих лабораторных работах).
- Освоить основы безопасности при работе с API-ключами (использование `.env` в сборке, ограничение прав ключа).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ (общий для всех вариантов)

Часть 1. Выбор стороннего API и подготовка проекта

1. **Выберите бесплатный (или тестовый) API** в соответствии с тематикой вашего курсового проекта. Примеры:
    - **Интернет-магазин / товары** → Fake Store API, DummyJSON Products

- **Новости** → NewsAPI, Currents API, GNews API
- **Рецепты** → TheMealDB, Spoonacular API
- **Фильмы** → The Movie Database (TMDb), OMDb API
- **Погода / геолокация** → OpenWeatherMap, WeatherAPI
- **Спорт** → TheSportsDB, Football-Data.org
- **Музыка** → Spotify Web API, Last.fm API
- **Книги** → Google Books API, Open Library API
- Если правильно выбрать тематику, то API можно использовать и в курсовом проекте.

2. **Зарегистрируйтесь** (при необходимости) и получите API-ключ (или используйте открытые эндпоинты без ключа).
3. **Переключитесь на новую ветку репозитория:**

   bash

```
git checkout main
git pull origin main
git checkout -b feature/async-api-storage
```

4. **Создайте структуру файлов для работы с данными:**

```
js/
├── api/
│   ├── apiService.js   — инкапсуляция запросов к API
│   └── config.js       — базовый URL, ключи (только для демонстрации; в production ключи не хранят на клиенте)
├── storage/
│   ├── localStorage.js — функции работы с LocalStorage
│   └── sessionStorage.js — (опционально)
├── utils/
│   └── dataParser.js    — вспомогательные функции преобразования данных
└── script.js            — основной модуль приложения
```

Часть 2. Реализация API сервиса

1. **В файле `js/api/config.js`** определите базовый URL и, при необходимости, API-ключ (в учебных целях можно хранить прямо в коде, но в реальном проекте используйте переменные окружения на сервере или ограничьте ключ по домену):

```
export const API_CONFIG = {
    baseUrl: 'https://api.example.com',
    apiKey: 'your_key_here',   // для демонстрации; лучше использовать переменную окружения
};
```

2. **В файле `js/api/apiService.js`** реализуйте функции для отправки запросов:
   - `fetchData(endpoint, params = {})` – универсальная GET-функция с использованием Fetch API.
   - Дополнительно можно реализовать `postData` при необходимости.

   Пример для Fake Store API:

```
import { API_CONFIG } from './config.js';

export async function getProducts() {
```

```
    const response = await fetch(`${API_CONFIG.baseUrl}/products`);
    if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`);
    return await response.json();
}

export async function getProductById(id) {
    const response = await fetch(`${API_CONFIG.baseUrl}/products/${id}`);
    if (!response.ok) throw new Error(`Product ${id} not found`);
    return await response.json();
}
```

3. **Добавьте обработку ошибок** (например, проверка статуса ответа, таймаут, отмена запроса через `AbortController`). Обратите внимание на возможные коды ошибок (404, 500 и т.д.).

Часть 3. Работа с клиентским хранилищем (LocalStorage)

1. **В файле `js/storage/localStorage.js`** создайте обёртки для сохранения и извлечения данных:

```
export function saveToLocalStorage(key, data) {
    try {
        localStorage.setItem(key, JSON.stringify(data));
    } catch (e) {
        console.error('LocalStorage save error:', e);
    }
}

export function loadFromLocalStorage(key) {
    try {
        const data = localStorage.getItem(key);
        return data ? JSON.parse(data) : null;
    } catch (e) {
        console.error('LocalStorage load error:', e);
        return null;
    }
}
```

2. **Определите список ключей** для хранения:
    - `'cached_products'` – кэш списка товаров (для офлайн-доступа).
    - `'favorites'` – избранные элементы.
    - `'cart'` – корзина.
    - `'search_history'` – история поиска.

Часть 4. Утилиты для работы с данными

1. **В файле `js/utils/dataParser.js`** создайте функции для преобразования данных из API в удобный для отображения формат (при необходимости):

```
export function parseProducts(rawProducts) {
    return rawProducts.map(p => ({
        id: p.id,
        name: p.title,
        price: p.price,
        image: p.image,
        description: p.description
    }));
}
```

Часть 5. Интеграция с UI и обработка состояний

1. **В основном файле `js/script.js`** реализуйте:
    - Загрузку данных из API при загрузке страницы (с индикатором загрузки).
    - Сохранение загруженных данных в `LocalStorage`.
    - Отображение данных (например, список товаров, новостей, рецептов) в HTML, созданном ранее.
    - Реализацию хотя бы одного интерактивного действия на основе API (например, поиск, фильтрация, добавление в избранное).
2. **Пример структуры `script.js`:**

```
import { getProducts } from './api/apiService.js';
import { saveToLocalStorage, loadFromLocalStorage } from './storage/localStorage.js';
import { renderProductList } from './ui/render.js';   // функция отображения данных (создайте её сами)

document.addEventListener('DOMContentLoaded', async () => {
    let products = loadFromLocalStorage('cached_products');
    if (!products) {
        showLoader();
        try {
            products = await getProducts();
            saveToLocalStorage('cached_products', products);
        } catch (error) {
            showError('Не удалось загрузить данные. Проверьте соединение.');
            return;
        } finally {
            hideLoader();
        }
    }
    renderProductList(products);
});
```

3. **Добавьте функциональность офлайн-доступа:** если API недоступно, использовать данные из `LocalStorage` (с пометкой «кэшированные данные»).

Часть 6. Реализация дополнительных требований (по выбору студента)

Выберите минимум два из следующих пунктов (в соответствии с вашим проектом):

- Реализовать поиск/фильтрацию через параметры API (например, `/products/search?q=...`).
- Добавить пагинацию (подгрузку следующей страницы).
- Сохранять историю поиска в `LocalStorage` и отображать последние запросы.
- Реализовать «избранное» (добавление/удаление, сохранение в `LocalStorage`).
- Реализовать корзину (добавление, удаление, подсчёт суммы) с синхронизацией в `LocalStorage`.
- Добавить индикатор «обновление данных» и кнопку «обновить» для принудительного запроса к API (игнорируя кэш).
- Использовать `AbortController` для отмены предыдущих запросов при быстром вводе поискового запроса.
- Реализовать retry (повтор запроса) при ошибке сети (например, до 3 раз с задержкой).

Часть 7. Обеспечение безопасности и обработка ошибок

- Убедитесь, что API-ключ не загружается в репозиторий (не публикуйте его). В учебном проекте достаточно хранить его в отдельном файле, который не коммитится (добавить в `.gitignore`), или использовать ограничение по домену в настройках ключа.

- Все сетевые запросы должны быть обёрнуты в `try...catch`.
- Предусмотрите отображение сообщений об ошибке для пользователя (например, «Сервер временно недоступен»).
- Для запросов, требующих авторизации, добавьте заголовок `Authorization: Bearer <token>` или `X-API-Key`.

Часть 8. Фиксация изменений

```
git add .
git commit -m "feat: integrate external API, caching and localStorage for [тип проекта]"
git push origin feature/async-api-storage
```

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист (по Стандарту Предприятия).
- Цель работы.
- Краткие теоретические сведения об асинхронном JavaScript, Fetch API и клиентских хранилищах.
- **Ход работы:**
    - Описание выбранного API и его соответствия теме курсового проекта.
    - Скриншоты:
        - Инструменты разработчика (вкладка Network) с выполненными запросами к API.
        - Данные в LocalStorage (вкладка Application) после сохранения.
        - Консоль браузера с выводами логов асинхронных операций (успех/ошибка).
        - Сообщения об ошибках при недоступности API (эмуляция офлайн).
        - Примеры запросов к API (можно скопировать из кода).
        - Фрагменты кода (ключевые функции: запрос, сохранение, отображение).
    - Ссылка на коммит с выполненной работой.
- **Ответы на контрольные вопросы** (все 30 вопросов).
- **Выводы** по работе.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- **Индивидуальное выполнение.** Все выполняют единый порядок действий (задание одинаково для всех вариантов, отличается только тематика проекта, которая выбирается в соответствии с будущим курсовым проектом).

!!!! Для выбора темы используется Генератор вариантов

- Код должен быть загружен в репозиторий GitHub в ветке `feature/async-api-storage`.
- В README репозитория укажите, какое API используется и как его настроить (если требуется ключ).

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы основные различия между `LocalStorage` и `SessionStorage`? В каких сценариях следует использовать каждый из них в вашем проекте?
2. Как механизм `Promise` улучшает работу с асинхронными операциями по сравнению с колбэками? Приведите пример из вашего кода.
3. Какие преимущества предоставляют `async/await` перед цепочками `then/catch`?
4. Какие стратегии обработки ошибок вы реализовали при работе с API? Как обеспечить отказоустойчивость приложения?
5. Какие методы кэширования данных вы применили? Как поддерживать актуальность кэшированных данных (инвалидация кэша)?
6. Как реализовать офлайн-функциональность? Какие данные наиболее критичны для кэширования в вашем проекте?
7. Что такое CORS и как он влияет на взаимодействие с API из браузера? Какие методы обхода CORS вы знаете?
8. Каковы лучшие практики безопасности при работе с API-ключами на клиенте? Как вы их применили?
9. Как реализовать пагинацию при работе с большими объёмами данных от API? Напишите пример кода.
10. Какие HTTP-статусы наиболее важны при работе с REST API? Как их правильно обрабатывать?
11. Как оптимизировать производительность при частых запросах к API (debounce, throttle, отмена предыдущих запросов)?
12. Что такое `AbortController` и как с его помощью отменить запрос при быстром поиске?
13. Как реализовать retry (повторную попытку) запроса при временной ошибке сети? Напишите пример.
14. Какие заголовки HTTP (`Content-Type`, `Authorization` и др.) вы использовали в запросах? Зачем они нужны?
15. В чём разница между методами `GET`, `POST`, `PUT`, `DELETE`? Когда какой следует применять при интеграции с API?
16. Как проверить, что ответ от API содержит ожидаемые данные? Какие проверки вы добавили?
17. Как смоделировать офлайн-режим в браузере для тестирования кэширования? (Chrome DevTools)
18. Какие есть ограничения `LocalStorage` (размер, типы данных, синхронность)? Как они влияют на ваш проект?
19. Как сохранить в `LocalStorage` не только данные, но и время их сохранения для реализации TTL (time-to-live) кэша?
20. Как с помощью `JSON.stringify` и `JSON.parse` корректно сохранять и восстанавливать объекты, содержащие даты или функции?
21. Как реализовать «ленивую загрузку» данных при скролле (infinite scroll) с использованием API?
22. Что такое «оптимистичное обновление» и как оно применяется при работе с API (например, добавление комментария)?
23. Какие инструменты для тестирования API (Postman, Insomnia) вы использовали при разработке?
24. Как обработать ситуацию, когда API возвращает ошибку `429 Too Many Requests`?
25. Что такое rate limiting и как его учитывать при разработке клиентского приложения?
26. Как реализовать «защиту от дублирования» одинаковых параллельных запросов?
27. Как передать параметры поиска, сортировки и пагинации в запросе к API (query string)? Приведите пример URL.

28. Как можно улучшить пользовательский опыт, показывая индикатор загрузки при запросе к API?
29. Что такое `Promise.all` и как его можно использовать для параллельной загрузки нескольких ресурсов?
30. Как вы организуете логирование ошибок на стороне клиента? (например, отправка в `console.error` или сохранение в `LocalStorage`).

---

8. ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Всегда обрабатывайте возможные ошибки при работе с API (используйте `try...catch`).
- Используйте индикаторы загрузки для улучшения UX (например, спиннер или текст «Загрузка...»).
- Ограничивайте размер данных в LocalStorage (обычно 5–10 МБ). Для больших объёмов используйте IndexedDB.
- Регулярно очищайте устаревшие данные из хранилища (например, при перезагрузке страницы или по истечении времени).
- Тестируйте приложение в условиях плохого соединения (вкладка Network → Throttling → Slow 3G).
- Используйте `AbortController` для отмены запросов при необходимости (например, при смене поискового запроса).
- Реализуйте retry-логику для неудачных запросов (не более 3 повторений с экспоненциальной задержкой).

---

9. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Если выбранное вами API требует платную подписку, можно использовать **Mock API** (например, `JSONPlaceholder` или `MockAPI.io`) с данными, соответствующими вашей тематике.
- Созданная в этой работе система работы с данными (API + LocalStorage) будет полезна в следующей лабораторной работе (№7 – взаимодействие React-приложения с сервером) и при разработке курсового проекта во 2 части дисциплины.